CME 307

LLM-Enhanced Course Scheduling

Erica Zhao* **and Kayla Li***
Department of Management Science & Engineering
Stanford, CA 94305, USA
{erica117,peizheli}@stanford.edu

Abstract

To address the opacity of traditional university timetabling, this study presents an LLM-Enhanced Course Scheduling system that integrates Mixed-Integer Linear Programming (MILP) optimization with a multi-agent Large Language Model interface for interactive decision support. The proposed architecture employs Gurobi for schedule generation while utilizing an X-MILP-inspired framework to perform counterfactual analysis and Irreducible Infeasible Subsystem (IIS) extraction, enabling transparent explanations for what-if scenarios. Evaluations on 150 synthetic instances demonstrate that student conflict density is the primary driver of computational complexity, identifying a critical performance threshold when average conflicts exceed four per student. By combining optimization with explainability and interactivity, this work bridges the gap between mathematical models and practical decision-making in university scheduling.

*These authors contributed equally.

CME307: Optimization, Autumn 2025, Stanford University, CA. (LateX template borrowed from NIPS 2017.)

1 Introduction

Universities must assign classes, instructors, and students to limited rooms and time slots under *hard constraints* that cannot be violated (e.g., no simultaneous instructor assignments, room capacity limits) and *soft constraints* representing preferences (e.g., avoiding evening slots, limiting back-to-back teaching) [4, 12]. Although mixed-integer programming (MIP) produces high-quality schedules, existing tools remain opaque: users rarely understand why specific assignments were made or why requests are infeasible [5]. Moreover, scheduling needs evolve as courses are added or preferences change, requiring flexible adjustment capabilities [8, 4].

To enables non-experts to interact through natural language, understand scheduling decisions, and explore adjustments, we propose LLM-Enhanced Course Scheduling, integrating (i) MIP-based timetabling with Gurobi, (ii) a multi-agent LLM interface for natural language explanations and constraint translation, and (iii) X-MILP-inspired counterfactual analysis using UDSPs and IIS extraction for what-if queries. We evaluate on 150 synthetic instances with systematically varied student conflict density ($\lambda \in \{1.5, \ldots, 3.5\}$, $\rho \in \{0.2, \ldots, 0.8\}$) to investigate how enrollment patterns impact computational complexity.

2 Literature Review

The university course timetabling problem (UCTTP) is NP-hard and extensively studied in operations research. Classical approaches employ integer programming [7] or metaheuristics such as Tabu Search and evolutionary algorithms [4], effectively handling large instances but offering limited interactivity or interpretability. Standard formulations distinguish hard constraints (resource conflicts, capacities, availability) from soft constraints (compactness, preferred time slots) [12, 5], with sensitivity analyses remaining technical and inaccessible to non-experts [8].

Recent advances demonstrate that LLMs can translate natural language into optimization models using multi-agent architectures [3, 13, 14], yet focus primarily on model generation rather than solver interpretation or interactive analysis. Complementing this, the X-MILP framework [13, 16] enables counterfactual reasoning through User-Desired Satisfiability Problems (UDSPs) and Irreducible Infeasible Subsystem (IIS) extraction, but has not been applied to large-scale timetabling or integrated with natural language interfaces.

Our work addresses these gaps by integrating MIP-based timetabling with multi-agent LLM interfaces and X-MILP-inspired explanations, enabling non-experts to express constraints naturally, receive human-interpretable solver explanations, and perform interactive counterfactual analysis.

3 Problem Setup

3.1 Model Definitions

We formulate the university course timetabling problem over courses $\mathcal{C}$, instructors $\mathcal{I}$, rooms $\mathcal{R}$, academic blocks $\mathcal{B} = \{1, 2\}$, days $\mathcal{D}$, and periods $\mathcal{P}$. The decision variable $x_{c,b,d,p,r} \in \{0, 1\}$ indicates whether course c is assigned to block b, day d, period p, and room r. Key parameters include course sessions $Sessions_c$, duration Dur_c, instructor assignments $Inst_c$, enrollment Enr_c, room capacity Cap_r, instructor availability $Avail_{i,d,p}$, and student conflicts $Conflict_{c,c'}$. Complete notation is provided in Appendix A.

3.2 Optimization Model

3.2.1 Objective Function

The objective minimizes the total weighted penalty for violations of soft constraints:

$$\min Z = Z_{S1} + Z_{S2} + Z_{S3} \tag{1}$$

3.2.2 Hard Constraints

Hard constraints are non-negotiable rules that must be satisfied for a timetable to be considered valid. Violation of any hard constraint would make the schedule unworkable.

C1. Instructor Conflict: An instructor cannot teach multiple courses simultaneously.

$$\sum_{c\in\mathcal{C}:Inst_c=i}\sum_{r\in\mathcal{R}}\sum_{\tau=\max(1,p-Dur_c+1)}^{p} x_{c,b,d,\tau,r} \leq 1 \quad \forall i\in\mathcal{I}, b\in\mathcal{B}, d\in\mathcal{D}, p\in\mathcal{P} \tag{2}$$

C2. Room Conflict: A room cannot host multiple courses simultaneously.

$$\sum_{c\in\mathcal{C}}\sum_{\tau=\max(1,p-Dur_c+1)}^{p} x_{c,b,d,\tau,r} \leq 1 \quad \forall r\in\mathcal{R}, b\in\mathcal{B}, d\in\mathcal{D}, p\in\mathcal{P} \tag{3}$$

C3. Session Frequency: Each course must be scheduled for exactly $Sessions_c$ times per week within active blocks.

$$\sum_{d\in\mathcal{D}}\sum_{p\in\mathcal{P}}\sum_{r\in\mathcal{R}} x_{c,b,d,p,r} = Sessions_c \quad \forall c\in\mathcal{C}, b\in\mathcal{B}: BlockActive_{c,b}=1 \tag{4}$$

C4. Instructor Availability: Courses are scheduled only during instructor availability windows.

$$x_{c,b,d,p,r} = 0 \quad \forall (c,b,d,p,r): \exists\tau\in[p,p+Dur_c-1] \text{ s.t. } Avail_{Inst_c,d,\tau}=0 \tag{5}$$

C5. Instructor Assignment: Each course is taught by its assigned instructor.

$$x_{c,b,d,p,r} \Rightarrow \text{instructor } Inst_c \text{ assigned} \quad \forall (c,b,d,p,r)\in\mathcal{X}_{valid} \tag{6}$$

C6. Time Window: All sessions occur within operational hours (8:00 AM – 8:00 PM).

$$p+Dur_c-1 \leq |\mathcal{P}| \quad \forall (c,b,d,p,r)\in\mathcal{X}_{valid} \tag{7}$$

C7. Room Capacity: Rooms must accommodate course enrollment.

$$x_{c,b,d,p,r} \Rightarrow Enr_c \leq Cap_r \quad \forall (c,b,d,p,r)\in\mathcal{X}_{valid} \tag{8}$$

C8. One Session Per Day: A course cannot have multiple sessions on the same day.

$$\sum_{p\in\mathcal{P}}\sum_{r\in\mathcal{R}} x_{c,b,d,p,r} \leq 1 \quad \forall c\in\mathcal{C}, b\in\mathcal{B}, d\in\mathcal{D} \tag{9}$$

C9. Schedule Consistency: Courses maintain identical weekly schedules within their active blocks. Full-term courses ($IsFullTerm_c = 1$) must have the same schedule in both blocks, while half-term courses maintain consistent schedules across all weeks within their designated block.

$$x_{c,1,d,p,r} = x_{c,2,d,p,r} \quad \forall c\in\mathcal{C}: IsFullTerm_c=1, \forall d\in\mathcal{D}, p\in\mathcal{P}, r\in\mathcal{R} \tag{10}$$

3.2.3 Soft Constraints

Soft constraints define schedule quality preferences, weighted by block duration to reflect cumulative impact over time. All soft constraint penalties are scaled by $Weight_b$, the number of weeks in block b, ensuring penalties reflect their cumulative impact across the academic term.

S1. Student Conflicts: Minimize overlapping class assignments for students.

Linearization: Define occupancy $Occ(c,b,d,p) = \sum_{r\in\mathcal{R}}\sum_{\tau=\max(1,p-Dur_c+1)}^{p} x_{c,b,d,\tau,r}$

$$Occ(c,b,d,p) + Occ(c',b,d,p) \leq 1 + \phi_{c,c',b,d,p} \quad \forall c<c', b, d, p \tag{11}$$

Penalty:

$$Z_{S1} = W_1 \sum_{c<c'}\sum_{b,d,p} Conflict_{c,c'} \cdot Weight_b \cdot \phi_{c,c',b,d,p} \tag{12}$$

S2. Instructor Compactness: Align teaching patterns with instructor preferences for consecutive sessions.

Let $T_{i,b,d}$ denote the number of sessions and $B_{i,b,d}$ the number of back-to-back pairs for instructor i on day d in block b. The compactness metric is $2B_{i,b,d} - T_{i,b,d} + 1$.

Penalty:

$$Z_{S2} = W_2 \sum_{i,b,d} Pref_i^{B2B} \cdot Weight_b \cdot (2B_{i,b,d} - T_{i,b,d} + 1) \cdot has_teaching_{i,b,d} \tag{13}$$

For $Pref_i^{B2B} = 1$ (prefers separation), the objective penalizes compactness. For $Pref_i^{B2B} = -1$ (prefers back-to-back), it rewards compactness.

S3. Lunch Avoidance: Minimize scheduling during lunch periods when instructors prefer free lunch.

Linearization:

$$Occ(c, b, d, p) \leq \pi_{c,b,d,p} \quad \forall c \in \mathcal{C}, b \in \mathcal{B}, d \in \mathcal{D}, p \in \mathcal{P}_{lunch} \tag{14}$$

Penalty:

$$Z_{S3} = W_3 \sum_{c,b,d} \sum_{p \in \mathcal{P}_{lunch}} Penalty_{Inst_c}^{Lunch} \cdot Weight_b \cdot \pi_{c,b,d,p} \tag{15}$$

4 Data

To evaluate solver scalability while preserving institutional privacy, we generate synthetic instances that simulate realistic university scheduling scenarios. Our stochastic generator models a mid-sized university with courses distributed across ten departments, with course levels weighted by probability (15% introductory, 35% upper-division) and term lengths (70% full-term, 30% half-term). Instructor profiles include availability windows and back-to-back teaching preferences $P \in \{-1, 0, 1\}$. The generator enforces pre-solve feasibility through capacity and availability constraints, with student enrollments correlated to academic level to create realistic conflict clustering (e.g., freshmen favor 100-level courses with probability 0.7).

The temporal domain discretizes into 30-minute periods across weekdays (08:00-20:00), yielding 120 time slots. We generated 150 instances with fixed size (50 courses, 30 instructors, 20 classrooms, 1000 students) but varying conflict structure, controlled by λ (average courses per student: 1.5-3.5) and ρ (conflict clustering coefficient: 0.2-0.8) to test the performance of our system. And generates another 75 instances to analyze how student conflict density impacts solver performance.

5 Methods

We develop a three-agent system (Figure 1): the Optimization Agent generates feasible schedules minimizing soft constraint penalties; the Explanation Agent provides natural language interpretations; the Counterfactual Analysis Agent enables "what-if" queries. Users configure constraints through a web interface (Appendix B) and view results via interactive calendar, chat, and analysis interfaces.

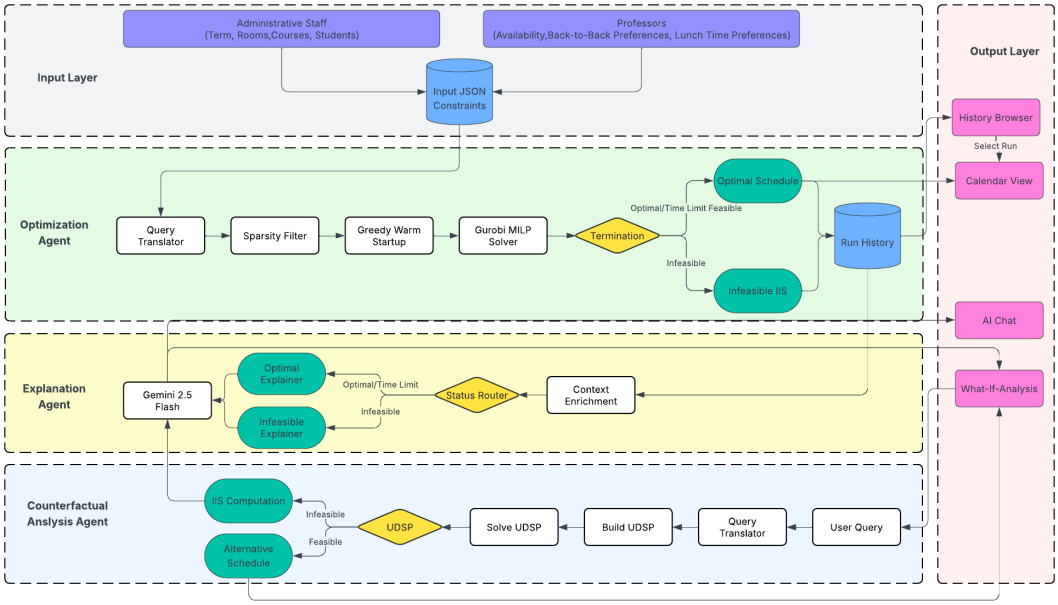


Figure 1: System architecture and data flow across the Input, Optimization, Explanation, Counterfactual, and UI layers.

5.1 Optimization Agent

We employ Gurobi 11.0 via Julia to solve the MILP formulation.

5.1.1 Model Sparsity and Symmetry Breaking

We construct a filtered variable space $\mathcal{X}_{\text{valid}}$ containing only feasible (c, b, d, p, r) tuples [2]:

$$\mathcal{X}_{\text{valid}} = \{(c, b, d, p, r) : \text{avail}_{i(c),d,p:p+\ell_c} \wedge \text{cap}_r \geq \text{enr}_c\} \tag{16}$$

c indexes courses, b indexes term blocks, d indexes days, p indexes periods, r indexes rooms, $i(c)$ is the instructor for course c, ℓ_c is session duration, and $\text{avail}_{i,d,p:p+\ell}$ indicates instructor availability.

For full-term courses spanning both blocks, we enforce symmetry-breaking constraints [11]:

$$x_{c,1,d,p,r} = x_{c,2,d,p,r} \quad \forall c \in \mathcal{C}_{\text{full}}, (c, 1, d, p, r) \in \mathcal{X}_{\text{valid}} \tag{17}$$

5.1.2 Greedy Warm Start

The greedy construction heuristic generates an initial feasible solution to accelerate MILP convergence [15, 1]. Courses are prioritized by enrollment size and term type. For each course, the algorithm assigns the earliest available time-room slot satisfying: (1) room capacity exceeds enrollment, (2) instructor availability covers all required consecutive periods, and (3) no conflicts exist with previously scheduled courses. The greedy solution is evaluated using the complete objective function.

5.2 Explanation Agent

We use Google's Gemini 2.5 Flash API to generate natural language explanations.

5.2.1 Context Enrichment

We transform solver output into structured context containing course metadata, instructor profiles, room specifications, and enrollment mappings, allowing the LLM to reference specific entities rather than indices.

5.2.2 Status-Dependent Strategies

For optimal solutions, we analyze soft constraint decomposition:

$$f^* = \sum_{i=1}^{n_1} w_1 \phi_i + \sum_{j=1}^{n_2} w_2 \beta_j + \sum_{k=1}^{n_3} w_3 \lambda_k \tag{18}$$

where ϕ_i, β_j, λ_k represent student conflict, instructor compactness, and time slot penalties. The agent computes concrete metrics and generates narratives explaining objective values in business terms.

For infeasible problems, we perform pre-analysis (total hours, room capacity, consecutive block availability) and synthesize the IIS with pre-analysis results to construct causal explanations like: "Course X requires 1.5 consecutive hours, but Instructor Y's slots are scattered across different days."

5.3 Counterfactual Analysis Agent

Following X-MILP [10], we formulate counterfactual reasoning as a User-Desired Satisfiability Problem (UDSP).

5.3.1 Query Translation

We support two primary constraint categories: *enforce* constraints requiring assignments not present in the optimal solution, and *veto* constraints prohibiting assignments present in the optimal solution.

For example, the query "Course c must be scheduled at week w, day d, period p" translates to:

$$\sum_{r \in \mathcal{R}} x_{c,w,d,p,r} = 1 \tag{19}$$

Conversely, a veto constraint "Course c cannot be scheduled on day d" becomes:

$$\sum_{w\in\mathcal{W}}\sum_{p\in\mathcal{P}}\sum_{r\in\mathcal{R}} x_{c,w,d,p,r} = 0 \tag{20}$$

5.3.2 UDSP Formulation

Given the original MILP $M = \langle f, \mathcal{C}\rangle$ with optimal solution x^* achieving objective value $f^* = f(x^*)$, and query constraints $\mathcal{C}_Q$, the UDSP is defined as:

$$\text{UDSP}(M, \mathcal{C}_Q) = \begin{cases} f(x) \le f^* & \text{(minimality)} \\ \mathcal{C} & \text{(hard constraints)} \\ \mathcal{C}_Q & \text{(query constraints)} \end{cases} \tag{21}$$

5.3.3 IIS Computation

For infeasible UDSP, we compute the Irreducible Infeasible Subsystem $\mathcal{I} \subseteq \mathcal{C} \cup \mathcal{C}_Q \cup \{f(x) \le f^*\}$ [6]. The IIS reveals whether infeasibility stems from hard constraint conflicts ($\mathbb{I}_{\text{min}} =$ false) or unacceptable objective degradation ($\mathbb{I}_{\text{min}} =$ true). For feasible UDSP, we return the alternative schedule with objective difference $\Delta f = f_{\text{alt}} - f^*$ decomposed into soft constraint changes $(\Delta S_1, \Delta S_2, \Delta S_3)$.

6 Theoretical Results

Theorem 1 (NP-Completeness). *The university course timetabling problem with hard constraints C1-C9 is NP-complete.*

Proof Sketch. Standard reduction from graph edge-coloring [9]: edges represent courses, vertices represent time slots, and instructor conflicts encode adjacency. Adding weighted soft constraints S1-S3 yields an NP-hard optimization problem. □

Model Reduction Properties. Our optimization techniques achieve substantial complexity reduction: (i) *Sparsity filtering* (Eq. 16) reduces variable space from $O(C \cdot B \cdot D \cdot P \cdot R)$ to $O(C \cdot D \cdot \alpha P \cdot \beta R)$ where $\alpha \approx 0.5$ (instructor availability) and $\beta \approx 0.3$ (room feasibility), yielding ≈15% of the original space. (ii) *Symmetry breaking* (Eq. 10) for full-term courses reduces search space by at least $2^{n_{\text{full}}}$ by enforcing identical block assignments. (iii) The *greedy heuristic* produces feasible solutions when they exist but provides no constant-factor approximation, where high-conflict course pairs can create arbitrarily large penalty ratios $Z_{\text{greedy}}/Z_{\text{opt}} \to \infty$.

Theorem 2 (Conflict Density Scaling). Let λ denote average courses per student and $\rho \in [0, 1]$ the conflict clustering coefficient. The expected number of conflict variables grows as:

$$\mathbb{E}[|\text{valid}_\phi|] \approx \rho \cdot \frac{\lambda^2}{2} \cdot S \cdot C_{\text{avg}} \tag{22}$$

where S is student count and $C_{\text{avg}} \approx 0.15 \cdot |B \cdot D \cdot P|$ is the average valid (block, day, period) combinations per course pair.

Proof Sketch. Each student enrolls in λ courses, generating $\binom{\lambda}{2} \approx \lambda^2/2$ potential conflict pairs. The clustering coefficient ρ determines what fraction of these materialize as actual conflicts. Each realized conflict pair creates indicator variables $\phi_{c,c',b,d,p}$ for overlapping (block, day, period) tuples, with C_{avg} capturing the average number of such valid combinations after feasibility filtering. □

This quadratic growth in λ suggests that student enrollment patterns may create a computational bottleneck as average course loads increase, motivating our experimental investigation of conflict density effects.

7 Numerical Results

All cleaned files, training logs, and documentation are in our Github repo: `https://github.com/Erica-zya/Course_Schedule_Optimizer.git`.

7.1 Student–conflict sensitivity experiment

This experiment examines how computational difficulty varies with student-course conflict structure (soft constraint S1) at fixed instance size: 1000 students, 50 courses, 30 instructors, 20 rooms, 10 weeks. Student enrollments are generated using two parameters: (i) λ (average courses per student), and (ii) $\rho \in [0, 1]$ (conflict clustering coefficient). Each student's course load K_s is drawn from a Poisson distribution with mean λ, truncated at six courses. Small ρ values produce sparse conflicts through uniform random selection, while large ρ values create dense conflicts via clustered course bundles. We test $\lambda \in \{1.5, 2, 2.5, 3, 3.5\}$ and $\rho \in \{0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8\}$ with five instances per (λ, ρ) pair, solving each with Gurobi and recording solve time and status. We measure conflict intensity as average conflicts per student (total student-course-pair conflicts divided by student count).

Figure 2 plots solve time ($\log_{10}$ seconds) versus average conflicts per student. Blue markers indicate successful solves; orange marks failures. We observed that a critical threshold emerges at 4-5 conflicts per student: instances below this exhibit short, stable solve times, while those above show substantially longer runtimes and increased failure rates. This demonstrates that student conflict density, rather than raw problem size, is the primary driver of computational complexity.

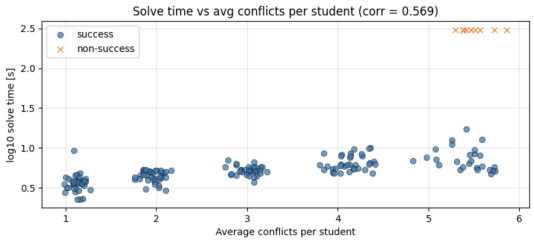


Figure 2: Solve time (log scale) versus average conflicts per student. Each marker represents one instance (blue: successful solve; orange: non-success).

To further investigate the interaction between model size and conflict structure, Figure 3 presents a heat map in which the horizontal axis corresponds to binned values of average conflicts per student and the vertical axis corresponds to binned estimates of the total number of MILP variables. Each cell contains the median solve time $\log_{10}$ over all instances that fall into the corresponding bin.

The heatmap indicates that cells with relatively low conflict tend to have shorter median solve times across a range of model sizes, while bins with higher conflict load display markedly larger median solve times. The slowest regimes occur where large variable counts coincide with high conflict density, indicating that dense conflicts are the main bottleneck for scaling the model to larger settings.

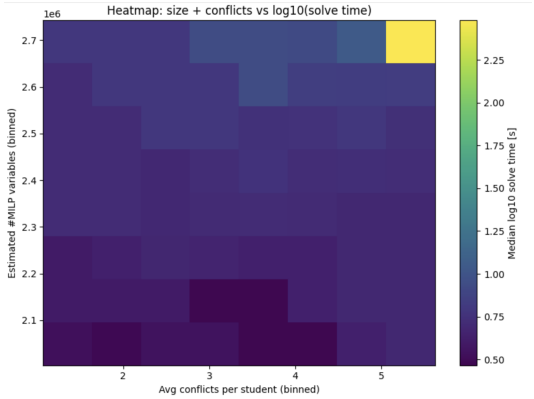


Figure 3: Heatmap of median $\log_{10}$ solve time as a function of estimated MILP size (vertical axis, binned) and average conflicts per student (horizontal axis, binned).

7.2 Optimization Progress

We evaluate convergence on a batch of large inputs (50 courses, 30 instructors, 20 rooms, and 1000 students over 14 weeks). For each instance, Gurobi records the initial heuristic penalty f_0 and the

incumbent objective $f(t)$ over 30 minutes. We convert these logs into a relative improvement measure

$$\text{reduction}(t) = 100 \cdot \frac{f_0 - f(t)}{f_0},$$

interpolate each run onto a common time grid, and then aggregate 150 runs.

The main pattern in figure 4 shows most of the work happens early. Within the first five minutes, the mean reduction already passes roughly 85–90%, and the median is very close to its final level. After that, the curves flatten. Extra time mainly shaves off small residual penalties: the final mean reduction is about 93.0%, with a 95% band of around ± 0.8 percentage points. The spread across runs is wide in the first minute, then shrinks, and the mean and median stay close throughout. This suggests that once the initial search has settled, the solver behaves quite consistently across different input files.

For staff and instructors, this has a direct practical reading. If the goal is to get a "good enough" timetable quickly, it is usually enough to let the solver run for 5 minutes as it has removed about 90% of the initial penalty. That corresponds to a modest time budget and is suitable for exploratory what–if sessions or weekly planning rounds.

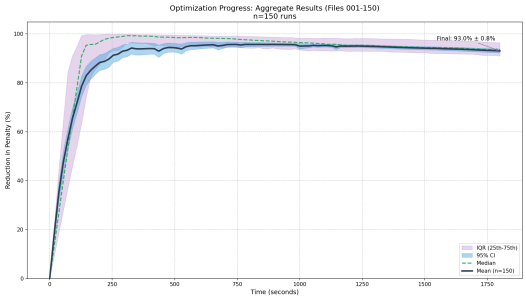


Figure 4: Aggregate optimization progress over 150 large-scale runs. Curves show percentage reduction in penalty relative to the initial heuristic solution over 30 minutes.

7.3 Limitations and Error Analysis

While our system successfully solves most instances, several limitations emerge. Solver failures occur in high-conflict regimes: 27.3% of instances failed to reach optimality when average conflicts exceeded 5 per student, where densely interconnected constraint networks overwhelm branch-and-bound heuristics. The quadratic growth in conflict variables (Theorem 2) suggests computational barriers when scaling beyond 100 courses or 2000 students.

Model assumptions limit practical applicability: (i) week-consistent scheduling (C9) prohibits mid-term adjustments for guest lectures or room maintenance, (ii) the static conflict matrix cannot capture dynamic enrollment changes during add/drop periods, and (iii) simple instructor preference values fail to capture nuanced requirements. LLM explanation quality suffers when IIS extraction produces minimal but unintuitive constraint sets, for example, identifying a capacity constraint as blocking when the true cause is a cascading availability conflict.

8 Conclusion and Future Work

We present an end-to-end course scheduling system integrating MILP optimization, LLM-based explanations, and counterfactual analysis. The system enforces institutional rules as hard constraints while minimizing soft penalties for student conflicts, instructor compactness, and lunch scheduling. Our optimization agent employs sparsity filtering, symmetry breaking, and greedy warm starts. The explanation agent transforms solver output into natural language narratives through context enrichment and status-dependent prompting strategies. The counterfactual agent enables what-if scenario testing via UDSP formulation and IIS computation.

In addition, the conflict-sensitivity study shows that, for fixed problem size, average conflicts per student are a dominant driver of solve time: once students face roughly four to five potential clashes, runtimes increase sharply and failures become more common. The aggregate progress analysis

further suggests that most of the penalty is removed in the first part of the run, with diminishing gains afterwards, which offers a practical knob for staff who want to trade off solution quality against computation time.

Though workable for now, our approach faces computational challenges when average student conflicts exceed 5 per student (27.3% failure rate in our experiments). The week-consistent scheduling assumption (C9) prevents mid-term adjustments, and the static conflict matrix cannot accommodate dynamic enrollment changes during add/drop periods. LLM explanations occasionally produce unintuitive constraint sets when IIS extraction returns minimal but non-obvious infeasibility causes.

Looking ahead, we have several directions to explore. First, implementing week-specific scheduling would enable staff to reschedule subsets of weeks (e.g., around holidays) while preserving the rest of the schedule, with the what-if module quantifying disruption costs. Second, the preference model could be expanded to cover student time-of-day preferences, program-level rules, and fairness constraints, possibly learned from historical schedules. Finally, on the algorithmic front, exploiting problem structure through decomposition methods and tightening the warm start heuristic via machine learning-guided prioritization could improve scalability.

9 Use of AI in this project

- **Idea generation.** AI was utilized to brainstorm potential intersections between mixed-integer programming and large language models. The final problem selection and system architecture design were determined by the authors.
- **Literature review.** AI assisted in locating some of relevant works on university timetabling, MILP-based scheduling, and explainable optimization. All cited papers were read and assessed manually to ensure technical relevance and correct interpretation.
- **Coding up an experiment.** AI was used to accelerate implementation of standard JuMP/Gurobi workflows and data-handling code. All constraint logic, objective design, and experimental control were written, tested, and validated by the authors.
- **Generating data to test on.** AI-supported scripting facilitated large-scale synthetic input generation. Enrollment distributions, instructor availability patterns, and feasibility checks were explicitly designed to reflect realistic university scheduling conditions.
- **Writing code for data visualization.** AI assistance was used for boilerplate plotting and aggregation routines. The choice of performance metrics, statistical summaries, and interpretation of solver behavior was carried out by the authors.
- **Writing the paper.** AI was used to improve clarity and organization. All mathematical formulations, experimental results, and substantive analysis were produced and revised by the authors.
- **Reviewing the paper.** AI was used as a proofreading aid to identify grammatical issues and unclear phrasing. All conceptual revisions and methodological judgments were made by the authors.

References

[1] Sina Abdipoor, Razali Yaakob, Say Leng Goh, and Salwani Abdullah. Meta-heuristic approaches for the university course timetabling problem. *Intelligent Systems with Applications*, 19:200253, 2023.

[2] Tobias Achterberg, Robert E Bixby, Zonghao Gu, Edward Rothberg, and Dieter Weninger. Presolve reductions in mixed integer programming. *INFORMS Journal on Computing*, 32(2):473–506, 2020.

[3] Ali AhmadiTeshnizi, Wenzhi Gao, and Madeleine Udell. Optimus: Optimization modeling using mip solvers and large language models. *arXiv preprint arXiv:2310.06116*, 2023.

[4] Hamed Babaei, Jaber Karimpour, and Amin Hadidi. A survey of approaches for university course timetabling problem. *Computers Industrial Engineering*, 86:43–59, 2015. Applications of Computational Intelligence and Fuzzy Logic to Manufacturing and Service Systems.

[5] Mei Ching Chen, San Nah Sze, Say Leng Goh, Nasser R. Sabar, and Graham Kendall. A survey of university course timetabling problem: Perspectives, trends and opportunities. *IEEE Access*, 9:106515–106529, 2021.

[6] John W Chinneck. *Feasibility and infeasibility in optimization: algorithms and computational methods*. Springer, 2008.

[7] Sophia Daskalaki and Theodore Birbas. Efficient solutions for a university timetabling problem through integer programming. *European journal of operational research*, 160(1):106–120, 2005.

[8] Nicholas G Hall and Marc E Posner. Sensitivity analysis for scheduling problems. *Journal of scheduling*, 7(1):49–83, 2004.

[9] Ian Holyer. The np-completeness of edge-coloring. *SIAM Journal on computing*, 10(4):718–720, 1981.

[10] Roger X Lera-Leri, Filippo Bistaffa, Athina Georgara, and Juan A Rodríguez-Aguilar. Exploiting constraint reasoning to build graphical explanations for mixed-integer linear programming. In *International Workshop on Explainable, Trustworthy, and Responsible AI and Multi-Agent Systems*, pages 21–39. Springer, 2025.

[11] François Margot. Symmetry in integer linear programming. *50 years of integer programming 1958-2008: from the early years to the state-of-the-art*, pages 647–686, 2009.

[12] Pedro Meseguer, Francesca Rossi, and Thomas Schiex. Chapter 9 - soft constraints. In Francesca Rossi, Peter van Beek, and Toby Walsh, editors, *Handbook of Constraint Programming*, volume 2 of *Foundations of Artificial Intelligence*, pages 281–328. Elsevier, 2006.

[13] Raghav Thind, Youran Sun, Ling Liang, and Haizhao Yang. Optimai: Optimization from natural language using llm-powered ai agents. *arXiv preprint arXiv:2504.16918*, 2025.

[14] Zhicheng Yang, Yiwei Wang, Yinya Huang, Zhijiang Guo, Wei Shi, Xiongwei Han, Liang Feng, Linqi Song, Xiaodan Liang, and Jing Tang. Optibench meets resocratic: Measure and improve llms for optimization modeling. *arXiv preprint arXiv:2407.09887*, 2024.

[15] Dezhen Zhang, Saijun Guo, Weishi Zhang, and Shujuan Yan. A novel greedy heuristic algorithm for university course timetabling problem. In *Proceeding of the 11th World Congress on Intelligent Control and Automation*, pages 5303–5308. IEEE, 2014.

[16] A. Zulkarnain, M. Amien, and R. F. Yusuf. Utilizing large language models (llm) for optimizing faculty schedule communication through google calendar integration. In *Proceedings of the 2025 International Conference on Computer Sciences, Engineering, and Technology Innovation (ICoCSETI)*, pages 762–767, Jakarta, Indonesia, 2025.

A Notation and Model Parameters

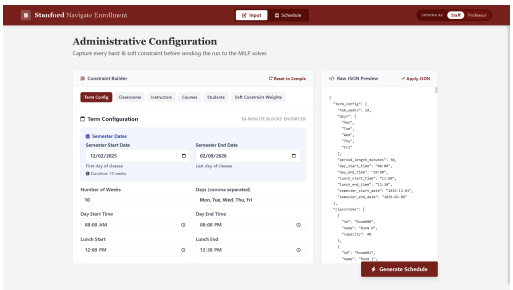
A.1 Sets and Indices

- $\mathcal{C}, \mathcal{I}, \mathcal{R}$: Courses, Instructors, Rooms
- $\mathcal{B} = \{1, 2\}$: Academic blocks (first half, second half)
- $\mathcal{D}, \mathcal{P}$: Days and periods
- $\mathcal{W}$: Weeks in academic term
- $\mathcal{X}_{valid}$: Valid assignment tuples (c, b, d, p, r)

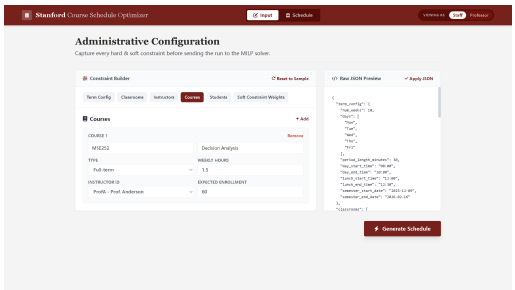
A.2 Parameters, Variables, and Definitions

Type	Symbol	Description
Params	$Sessions_c, Dur_c$	Weekly sessions and duration (periods) for course c
	$BlockActive_{c,b} \in \{0, 1\}$	1 if course c is active in block b
	$IsFullTerm_c \in \{0, 1\}$	1 if course c runs for full term (both blocks)
	$Inst_c \in \mathcal{I}$	Assigned instructor for course c
	$Enr_c \in \mathbb{Z}^+$	Expected enrollment for course c
	$Cap_r \in \mathbb{Z}^+$	Room capacity
	$Avail_{i,d,p} \in \{0, 1\}$	1 if instructor i is available at day d, period p
	$Conflict_{c,c'} \in \mathbb{Z}_{\geq 0}$	Number of students enrolled in both courses
	$Weight_b \in \mathbb{Z}^+$	Number of weeks in block b
	$W_{1,2,3} \in \mathbb{R}^+$	Objective weights for soft constraints
	$Pref_i^{B2B} \in \{-1, 0, 1\}$	Back-to-back preference (1: dislike, -1: like, 0: neutral)
	$Penalty_i^{Lunch} \in \{0, 1\}$	Lunch teaching penalty (0 if allowed, 1 if discouraged)
	$\mathcal{P}_{lunch} \subseteq \mathcal{P}$	Set of lunch periods
Decision Vars	$x_{c,b,d,p,r} \in \{0, 1\}$	1 if course c assigned to (b, d, p, r)
Auxiliary Vars	$\phi_{c,c',b,d,p} \in \{0, 1\}$	1 if courses c, c' conflict at (b, d, p)
	$h_{i,b,d,p} \in \{0, 1\}$	1 if instructor i teaching at (b, d, p)
	$\pi_{c,b,d,p} \in \{0, 1\}$	1 if course c occupies lunch period p at (b, d)
	$has_teaching_{i,b,d} \in \{0, 1\}$	1 if instructor i has any session at (b, d)
	$B_{i,b,d} \in \mathbb{Z}_{\geq 0}$	Number of back-to-back session pairs
	$T_{i,b,d} \in \mathbb{Z}_{\geq 0}$	Number of sessions for instructor i at (b, d)

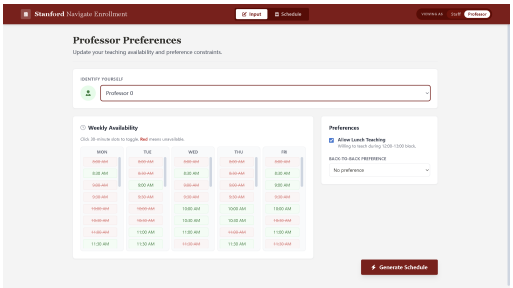
B User Interface Screenshots Examples



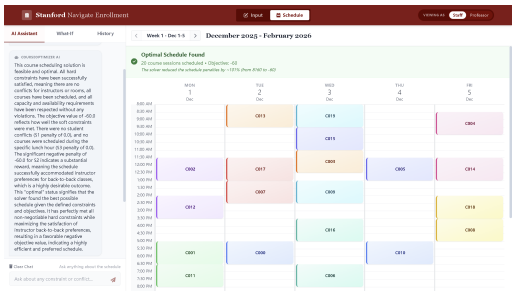
(a) Administrative configuration



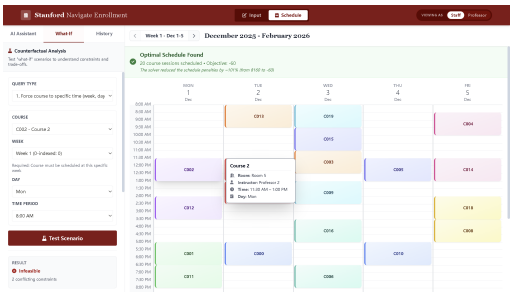
(b) Course assignment interface



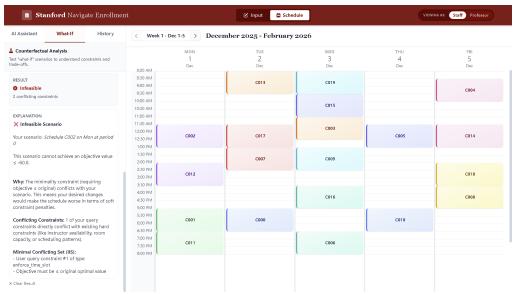
(c) Professor availability grid



(d) Weekly calendar view



(e) What-if analysis panel



(f) AI chat interface for what-if analysis

Figure 5: System user interfaces.